Reminder 学习 · 第5项 · 实证篇

Pi Durable 断了如何接上?

从SQLite命名空间，到压缩系统基线，再到进度落盘与睡眠唤醒。

不是“永不丢进度”，而是“明确哪些已承诺”。

真实完成范围

【我实际运行过（云端隔离环境）】6个文件、245个测试通过；其中2个是本篇新增实验，243个是上游测试。SQLite是真实Node内置SQLite，模型是本地faux。

> 【未验证】用户本机0.84.2、付费模型、Cloudflare生产部署、真实断电/强杀。本篇不修改reminder、不建日程、不碰vault/Git发布/账户配置。

固定源码：b30d54b8aebe6408e02d1473446d25f8083da68f
版本：pi-durable 1.5.2；提交：北京时间 2026-11-11 00:38；读取：北京时间 2026-11-11。

原帖：2112456455123022031（北京时间2026-10-20 16:10:41）；引用长文基于1.1.0。清单里的“久未查看”不是帖子的发布时间。

4种用途（应用建议，不是已部署）

① 长研究任务中断恢复；② 低频后台等待与宿主唤醒；③ 同一SQLite内多套Agent命名空间；④ 可回溯历史与压缩上下文的分层管理。都必须另设权限、幂等与运维边界。

来源状态分四类：作者说法 / 源码证实 / 我实际运行过（云端隔离环境） / 未验证。没有自称独立审核通过。

从浅到深 · 1—2

Q01 Pi Durable 是升级后的 Pi 命令行吗?

【源码证实】不是。它是供开发者嵌入的持久化 Agent 运行库，不是给现有终端加一个开关。Harness 管存储与调度，Conversation 管记录，Task 管可恢复状态机，Document 管应用状态。二者共用模型接入组件，不代表用户的终端自动获得 Durable 能力。



证据：核心概念 · 包与依赖

Q02 0.84.2 能直接用本文配置吗?

【源码证实】不能当作内置功能使用。用户本机安装的是 pi-coding-agent 0.84.2，不是 pi-durable 1.5.2；旧版依赖没有 pi-durable。本篇固定在新库源码上实验，需要独立 Node 项目，不升级也不修改用户机器。旧会话向 Durable 迁移、CLI 扩展桥接均【未验证】。



证据：0.84.2依赖声明 · 包与依赖

从浅到深 · 3—4

Q03 断点恢复究竟保存什么?

【源码证实】保存的是已提交记录、Document 和任务 checkpoint，不是 CPU 指令或整个进程内存。重开同一存储后 resume() 推进未完成任务；运行阶段可能从顶部重入，所以外部写入不能无条件重试。【我实际运行过（云端隔离环境）】官方恢复测试包含流式中断与压缩重开，不是物理断电测试。



证据：恢复与提交去重 · 进度丢失与恢复测试

Q04 两个实例怎么共享同一 SQLite?

【源码证实】prefix 会加在所有 Durable 表和索引名字前，不只是会话 ID 上。alpha_ 与 beta_ 形成各自 schema/metadata/records 空间。【我实际运行过（云端隔离环境）】同一文件两存储都分配 ID=2，读到各自不同内容，18 张表，重开 beta_ 仍为 BETA ONLY。



证据：表与索引前缀实现 · Node前缀验证

从浅到深 · 5—6

Q05 前缀隔离等于安全多租户吗?

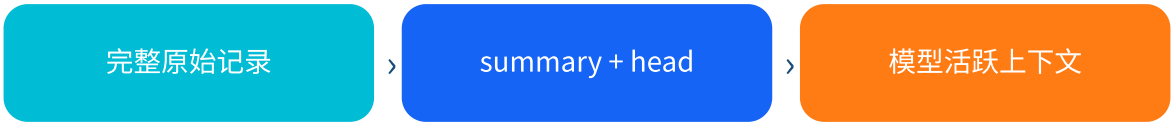
【源码证实】不是。它是命名空间隔离；持有同一文件访问权限的人仍可直接读别的表。Node 打开前会校验 prefix，避免非法标识符；同一个 prefix 不应用于不协调的独立写入者。本实验只覆盖同进程两个存储，不覆盖多进程争用、授权或加密。



证据：Node前缀验证 · 前缀格式校验

Q06 压缩会把旧记录删掉吗?

【源码证实】不是。压缩增加 summary/head 标记，改变发给模型的活跃上下文，原始不可变记录仍在。【我实际运行过（云端隔离环境）】真实库+faux 摘要的回归测试检查 before 等于 after 的原历史前缀，且模型下一请求含摘要和保留轮次；这只证明组织流程，不证明真实摘要语义无损。

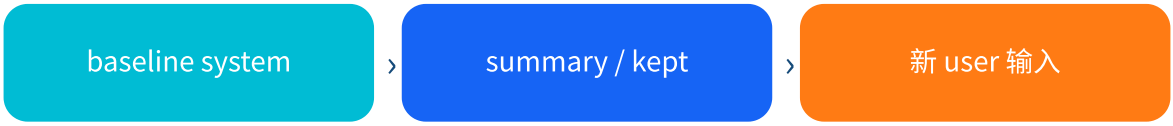


证据：真实压缩回归断言

从浅到深 · 7—8

Q07 System Message 顺序修复在哪里?

【源码证实】findBaseline 找到 head 之后完整的 pi.system；leadWithSystem 只把初始或基线系统消息前置，不把后续所有 system 一股脑前移。压缩保留 assistant 时旧逻辑可能让基线留在中途。【我实际运行过（云端隔离环境）】官方回归精确要求 roles=system,user,user,assistant,user，测试通过。



证据：baseline选择与前置 · 真实压缩回归断言

Q08 帖子说默认100ms，现在仍一样吗?

【作者说法】更新帖称进度提交间隔默认100ms。【源码证实】当前有两层：partialIntervalMs 与 outputIntervalMs 默认100ms；persistWithinMs 默认1000ms。1.5.0将内存可见进度与落盘延迟分离，因此不能把“100ms一次进度”解释为“100ms必落盘”。

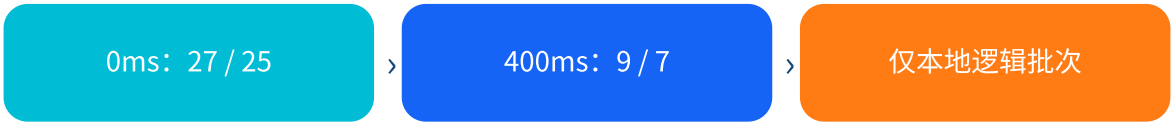


证据：当前进度默认值 · 1.5.0延迟持久化

从浅到深 · 9—10

Q09 调大间隔到底省了多少?

【我实际运行过（云端隔离环境）】相同640字符faux流、100ms partial节流，persistWithinMs=0时27批提交/25批含文档改变；400时9批/7批。两次都 done。此处计量是 ControlledStorage 的逻辑批次，不是远端请求数、磁盘IOPS、收费金额或作者的性能倍数，实际部署需另测。



证据：只注册faux模型 · 进度丢失与恢复测试

Q10 正常关闭与崩溃会损失同样进度吗?

【源码证实】正常 close 会在关闭存储前刷出 held documents；failed Session 不再补写。【我实际运行过（云端隔离环境）】官方测试验证 clean close 保存 held partial；模拟 fail 则仅从已存内容恢复，并可能生成 aborted entry。测试通过不等于宿主强制休眠一定调用close；这部分【未验证】。



证据：close提交延迟状态 · 进度丢失与恢复测试

从浅到深 · 11—12

Q11 休眠是不是不用任何云资源?

【作者说法】帖子说待机不耗额外资源。【源码证实】sleeping task 不占一次活跃调用；onWake 把最早截止时间交给宿主、避免内部 sleep timer。宿主仍需持久存储、alarm 与重新唤醒执行，不能据此宣称零成本。【我实际运行过（云端隔离环境）】sleep/onWake 单测通过，未部署Cloudflare。



证据：持久睡眠 / 宿主唤醒 · 1.4.0睡眠与迁移

Q12 恢复是否保证工具恰好执行一次?

【源码证实】不保证外部效果 exactly-once。提交 requestId 可去重输入，但外部发送与本地提交之间存在故障窗。写工具应有幂等键、可核对回执和明确重放策略。只在自有测试环境注入故障；真实账号、用户reminder与付费模型均【未验证】，本篇不创建真实通知或日程。



证据：恢复与提交去重 · 阶段重入边界

主实验 • 准备与隔离

【我实际运行过（云端隔离环境）】一次顺序全链运行覆盖以下步骤；子命令供按需重跑，不冒称每条过滤命令另跑过。运行目录：解压后的exercise/。

1. 固定输入、查重与版本

目的：防止把当前文档套到旧CLI。

```
git -C ../repo rev-parse HEAD
git -C ../repo log -1 --format="%H %cI"
node --version
```

真实结果：b30d54b8aebe6408e02d1473446d25f8083da68f；Node v24.22.1。提交时间转换为北京时间：2026-11-11 00:38。
验收：SHA与sources一致；旧0.84.2仅对照不安装。

2. 准备隔离依赖和源码解析

目的：依赖只在隔离目录；bootstrap会执行整个测试链。

```
npm ci --ignore-scripts --no-audit --no-fund
# 完整安装/源码链接命令已保存在本包：
bash ma-laoshi-20261111-bootstrap.sh
```

真实结果：本轮初次npm install实装131个包，用时6秒，未执行生命周期脚本。初跑83测试通过、4套件解析失败；映射修正后全过，失败日志保留。
验收：package-lock锁依赖；最终config精确匹配包名，源码未改。

3. 两个prefix共用真实SQLite

目的：验证不是Python仿制表，而是运行上游SqliteStorage。

```
node_modules/.bin/vitest run --config ma-laoshi-20261111-vitest.config.ts -t "two prefixes" --reporter verbose
```

真实结果：首个与后续全链运行实际执行同名测试：aId=bId=2，18张表；隔离与重开断言通过。
验收：alpha读ALPHA ONLY；beta先无对应记录，写入后重开BETA ONLY。

主实验 • 压缩、写入与验收

【我实际运行过（云端隔离环境）】一次顺序全链运行覆盖以下步骤；子命令供按需重跑，不冒称每条过滤命令另跑过。运行目录：解压后的exercise/。

4. 压缩、上下文与睡眠回归

目的：真实运行压缩后system置前、raw history不删、sleep/onWake行为。

```
node_modules/.bin/vitest run --config ma-laoshi-20261111-vitest.config.ts repo/packages/durable/test/harness-compaction.test.ts repo/packages/durable/test/harness-context.test.ts repo/packages/durable/test/harness-sleep.test.ts --reporter verbose
```

真实结果：同一次顺序全链中这些文件全部通过；压缩后下一请求角色序列被官方断言逐项比较。
验收：faux模型不发付费请求；sleep为受控时钟/运行时测试，不是Cloudflare实测。

5. 改变持久化窗并测批次数

目的：同样faux流对比0与400ms；避免引用作者性能当自己测量。

```
node_modules/.bin/vitest run --config ma-laoshi-20261111-vitest.config.ts -t "measure committed" --reporter verbose
```

真实结果：实际全链同名测试输出：0ms总27批/文档25批；400ms总9批/文档7批；两者done。
验收：断言第二组文档批次更少；不推断真实网络写入与费用。

6. 全链验收并保存故障边界

目的：覆盖SQLite恢复/中断/正常close/压缩/休眠，再核对全部结果。

```
node_modules/.bin/vitest run --config ma-laoshi-20261111-vitest.config.ts --reporter verbose
# 日志文件：../raw/ma-laoshi-20261111-tests-final.log
```

真实结果：6文件、245测试通过，首次成功耗时14.85秒；最终可移植配置再次运行结果见原始日志。
验收：不包含真实SIGKILL、断电、跨进程争用、真实模型质量和生产宿主计费。

版本地图 · 只补新增知识

阶段	核查结论
CLI 0.84.2	旧成品依赖无pi-durable；不能套用本篇SDK。
Durable 1.0.3	progress.partialIntervalMs/outputIntervalMs可配置，默认100ms。
1.3.0	prefix与压缩后baseline顺序修复。
1.4.0	sleepUntil/onWake；SQLite迁移v2，不能回滚到≤1.3.0。
1.5.0 → 当前1.5.2	deferred persistence默认1000ms；Storage.commit批次接口等破坏性变更。

1.0.3进度间隔 · 1.3.0更新 · 1.4.0睡眠与迁移 · 1.5.0延迟持久化

> 【作者说法】原帖已提休眠，但帖时间为10月20日；【源码证实】正式1.4.0变更日志日期为10月21日。不能把该帖作为“10月20日正式发布已可用”的证明。

查重与继承

先读公开main总览和旧笔记，固定 a8674777e18302b8804b137212b1396cbfe5a747。复用《Checkpoint存档与持久执行》的“恢复粒度≠外部exactly-once”勘误；复用《Compaction上下文压缩算法》的背景，不重复它的CLI算法。新增：表/索引命名空间、baseline前置、进度内存/存储双层、宿主唤醒契约。

风格参照旧 fast-jev-compaction 指南：编号疑问、彩色卡片、大号重点、SVG与实验步骤；本篇使用青蓝橙，不嵌字体与大图。

来源与未验证边界

原帖note_tweet全文、引用长文plain_text已成功读取并存raw；原帖讨论串两页读完，仅见两条非作者回复，未见作者回复。长文媒体/内链未全追读；视频未观看，不据视频新增结论。

实现证据：baseline选择与前置 · 表与索引前缀实现 · 当前进度默认值 · close提交延迟状态
实验证据：真实压缩回归断言 · 进度丢失与恢复测试 · 只注册faux模型
完整URL/日期/SHA：ma-laoshi-20261111-sources.md；完整输出：raw/内各.log。

本报告为学习候选成果，尚未合入共享知识库。所有测试由本worker执行，不伪造第三方独立审核。